

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor $g[i]$, which is the minimum size of a cookie that the child will be content with; and each cookie j has a size $s[j]$. If $s[j] \geq g[i]$, we can assign the cookie j to the child i , and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input:

3
1 2 3
2
1 1

Output:

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Constraints:

$1 \leq g.length \leq 3 * 10^4$

$0 \leq s.length \leq 3 * 10^4$

$1 \leq g[i], s[j] \leq 2^{31} - 1$

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int compare(const void *a, const void *b) {
4     long diff = (long)*(int *)a - (long)*(int *)b;
5     if (diff < 0) return -1;
6     if (diff > 0) return 1;
7     return 0;
8 }
9 int main() {
10     int n, m;
11     if (scanf("%d", &n) != 1) return 0;
12     int *g = (int *)malloc(n * sizeof(int));
13     for (int i = 0; i < n; i++) {
14         scanf("%d", &g[i]);
15     }
16     if (scanf("%d", &m) != 1) {
17         free(g);
18         return 0;
19     }
20     int *s = (int *)malloc(m * sizeof(int));
21     for (int i = 0; i < m; i++) {
22         scanf("%d", &s[i]);
23     }
24     qsort(g, n, sizeof(int), compare);
25     qsort(s, m, sizeof(int), compare);
26     int child_ptr = 0;
27     int cookie_ptr = 0;
28     while (child_ptr < n && cookie_ptr < m) {
29         if (s[cookie_ptr] >= g[child_ptr]) {
30             child_ptr++;
31         }
32         cookie_ptr++;
33     }
34
35     printf("%d\n", child_ptr);
36     free(g);
37     free(s);
38     return 0;
39 }
```

	Input	Expected	Got	
✓	2	2	2	✓
	1 2			
	3			
	1 2 3			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.